HANDS-ON GUIDE

# Testing the Orbital Voice Agent

What to set up, what to say, which tool each phrase triggers, and where to look to confirm that every feature works: the six assessment scenarios, every tool, every page, and the automated suites.

AUDIENCE

Anyone demonstrating or reviewing the application

TIME NEEDED

About 20 minutes for the full walkthrough

COST

Roughly $0.005 per spoken turn, $0.001 per typed turn (OpenAI mode)

DOCUMENT DATE

10 November 2026

# Contents


1. Before you start
2. Two modes: OpenAI and mock
3. The screen, explained
4. Voice flows
5. The six assessment scenarios
6. Tool-by-tool checks
7. Tools & tickets page
8. Knowledge base page
9. Conversations page
10. API and Swagger checks
11. Automated tests
12. Troubleshooting


# 1. Before you start

1. **Docker Desktop is running.** The API needs Postgres (host port 5434) and Redis (6379); `pnpm bootstrap` starts both and `pnpm dev` checks them.
2. **The key is in `.env`.** `OPENAI_API_KEY=sk-…` at the repository root. With it, the API reports `"provider": "openai"` at `http://localhost:4000/api/health`; without it, `"mock"`.
3. **Start everything:** `pnpm dev` (or `pnpm bootstrap` once on a fresh clone, then `pnpm dev`). Open `http://localhost:3000` in Chrome, Edge or Safari 16.4+.
4. **Allow the microphone** when the browser asks, after clicking the orb the first time. Use headphones or keep the volume moderate: the agent's own voice through speakers can trigger the barge-in detector if echo cancellation struggles.
5. **Pick a persona** (top right). Alice in Engineering has 11 of 20 PTO days left; Bob in Sales has 7 of 25; Carla in People Operations has 17 of 20. Balance questions and tickets use the selected persona automatically.

> The header shows two badges: the connection state (**connected** once the socket session is ready) and the provider (**OPENAI** or **MOCK**). Hover the provider badge for what the mode means.

## 2. Two modes: OpenAI and mock

| | OPENAI MODE | MOCK MODE (NO KEY) |
|---|---|---|
| Understands speech | Yes, real transcription | No: replays six demo utterances in order, one per spoken turn |
| Understands questions | Yes | Keyword planner: handles the demo phrases, not free-form questions |
| Voice | Natural speech, streamed | A short chime whose length follows the reply |
| Tools, citations, trace, tickets | All real | All real (same code), deterministic |
| Cost | ≈ $0.005 per spoken turn | Nothing |

Everything in this guide is written for OpenAI mode. In mock mode use the text box for your own questions and expect the voice path to transcribe one of the demo utterances regardless of what you said.

## 3. The screen, explained

| AREA | WHAT IT SHOWS | WHAT TO CHECK |
|---|---|---|
| Transcript (centre) | Your utterances on the right, answers on the left. Blue chips like `1` are citations; the list under an answer names the cited documents. | Click a chip: the sources sheet opens on that passage with its score. |
| Orb (bottom) | Black = idle · green pulse = listening (ring grows with your voice level) · amber spinner = transcribing / thinking / using tools · blue bars = speaking. | The label under it names the phase. |
| Agent trace (right) | Every step as it happens: pre-fetched search, tool calls with arguments (expand a row), results with durations, the retrieval grade badge, "Answer ready". | Timestamps are relative to the start of the turn. |
| Latency and cost badges (bottom right) | STT · 1st token · 1st audio · Total · cost ≈ $, plus model/tool call counts. | Hover the cost badge for token counts (prompt, cached, completion). |
| Controls (top) | Persona, Hands-free / Push to talk, Reset, Trace (on small screens). | Reset starts a fresh conversation; the old one stays in history. |

# 4. Voice flows

## 4.1 Hands-free (default)

**Click the orb once. Say a question and stop talking.**
**Expect:** The orb turns green while you speak; ~0.7 s after you stop it turns amber, your words appear on the right, the trace fills, the answer streams in and is spoken. The orb returns to green (still listening) when playback ends.

**Ask a second question without touching anything.**
**Expect:** It is picked up the same way: hands-free stays on until you click the orb again.

**Barge-in: while the agent is speaking, start talking.**
**Expect:** Playback stops within a word or two, the trace shows the turn was cancelled, and your new utterance is handled.

## 4.2 Push to talk

**Switch to Push to talk. Press and hold the orb, speak, release.**
**Expect:** Only what you said while holding is sent; the label reads "Release to send" while held.

## 4.3 Text fallback

**Type in "Or type a question..." and press Enter.**
**Expect:** Same pipeline, same trace, spoken reply.

## 4.4 Stop and reset

**Click Stop during a long answer.**
**Expect:** Speech and generation stop; the partial answer is kept.

**Click Reset.**
**Expect:** Empty transcript, "Say something" placeholder, a new conversation id (the previous one is under Conversations).

# 5. The six assessment scenarios

Say each phrase (or type it). The "look for" column tells you how to prove the behaviour, not just see an answer.

| # | SCENARIO | SAY | LOOK FOR |
|---|---|---|---|
| 1 | Voice conversation | "What are the core working hours for remote employees?" | Full voice round-trip; STT badge filled; answer cites the Remote Work Policy or Handbook. |
| 2 | Knowledge-grounded answer | "How many vacation days do new employees get, and can I carry unused days over?" | Chips [1] [2]…; sources sheet shows PTO and Leave Policy › Allowance and › Carryover; trace shows the pre-fetched search graded *strong*. |
| 3 | Semantic retrieval | "How long do I have to claim back money I spent on a business trip?" | The policy says "submitted within 30 days": no shared words with the question, yet it is found and cited. |
| 4 | Agentic execution | "I'd like to take off from December 21st through the end of the year. How many PTO days would that cost me, and do I have enough left?" | Several steps in the trace: pre-fetched search, then `search_knowledge_base` and `get_leave_balance` together (and `count_business_days` / `calculate` when used). Answer: six PTO days, eleven remaining for Alice. |
| 4b | Confirmed action | "My laptop screen is cracked, what should I do?" then "Yes please, high priority" | First answer quotes the equipment policy and offers to open a ticket; second runs `create_support_ticket` and reads out a TCK- reference. Open Tools & tickets: the ticket is there, high priority, linked to the conversation. Then ask "What's the status of my tickets?": `get_support_tickets` fetches them over HTTP and the agent reads back reference and status. |
| 5 | Contextual follow-up | After #2: "And what about sick days?" | Answered from the same policy without asking what you mean; the history is in the prompt. |
| 6 | Insufficient information | "What's the policy on bringing dogs to the office?" | Trace: pre-fetch graded *weak*, a second search with new phrasing, then an honest "the knowledge base doesn't cover this" with a pointer to People Operations and an offer to open a ticket. No citations. |

> Also try the persona switch: select Bob, ask "How much vacation do I have left?" and expect seven of twenty-five days; nobody is ever asked for an employee id.

# 6. Tool-by-tool checks

| TOOL | SAY | EXPECT IN THE TRACE | WHERE THE EFFECT IS VISIBLE |
|---|---|---|---|
| `search_knowledge_base` | Any policy question, e.g. "Do I need a receipt for a $20 taxi?" | "Searching the knowledge base" with queries; result "N passages (strong)"; grade badge | Citation chips, sources sheet, Knowledge page search for the same query |
| `get_current_datetime` | "Is there a company holiday coming up this month?" | "Checking the date and time" → "Monday, 10 November 2026 … (America/New_York)" | Answer reasons from today's date |
| `count_business_days` | "If I take December 21st to the 31st off, how many PTO days is that?" | "Counting working days" → "9 weekdays − 3 holidays = 6 business days" | Answer says six |
| `calculate` | "I've used seven days this year; how many are left if I take five more?" | "Calculating" → "20 - 7 - 5 = 8" | Answer says eight |
| `get_leave_balance` | "How many PTO days do I have left?" | "Looking up leave balance" → "Alice Nguyen: 11 of 20 PTO days left …" (~150 ms simulated HR latency) | HR cards on the Tools page show the same numbers |
| `get_support_tickets` | "What's the status of my tickets?" or "Any update on TCK-…?" | "Checking your support tickets" → "2 tickets for Alice Nguyen (1 open, 1 resolved)"; the result row expands to show the HTTP-fetched tickets | Matches the Tools & tickets table; change a status there and ask again |
| `create_support_ticket` | "My phone was stolen" → "Yes, open it as urgent" | Offer first; after yes: "Creating a support ticket" → "Ticket TCK-XXXXXX created (urgent priority)" | Tools & tickets table; `GET /api/hr/tickets`; conversation link |

Expand any trace row (chevron) to see the exact arguments the model sent and, for searches, the passages with their scores. Failed tools show a red mark with the error; the agent recovers in the next step.

## 7. Tools & tickets page

**Open Tools & tickets in the sidebar.**
**Expect:** Four sections: Support tickets, Tool activity, Tool catalogue (seven cards, each with its argument schema and a "Try:" phrase), HR system (three personas with PTO and sick meters). The page refreshes every few seconds.

**Create a ticket by voice in another tab, then come back.**
**Expect:** The new row appears within ~8 seconds: reference, title, priority badge, requester id, "View conversation" link.

**Change a ticket's status from Open to In progress or Resolved.**
**Expect:** Toast "TCK-... marked resolved"; the row updates; `GET /api/hr/tickets` reflects it.

**Read the Tool activity bars.**
**Expect:** Calls per tool with failure counts and average duration; the recent-invocations table links each result to its conversation.

## 8. Knowledge base page

**Search "What happens to my unused vacation at the end of the year?" in Hybrid mode.**
**Expect:** Ranked passages with three score bars (fused, dense, keyword); the PTO policy's Carryover section first.

**Switch to Dense and Keyword.**
**Expect:** Orderings differ; Keyword shows BM25-only ranks, Dense embedding-only.

**Click a document in the left list.**
**Expect:** A sheet with the rendered document, its category, chunk count and sections.

**Click Reindex all.**
**Expect:** The button counts jobs ("Indexing 3/12 ...") as the BullMQ worker processes them; a success toast at the end.

## 9. Conversations page

**Open Conversations.**
**Expect:** Every session with title (first question), persona, message count and last activity.

**Open one.**
**Expect:** Full transcript with citation chips and the last turn's latency/cost badges. "Continue by voice" reopens it in the agent with the same history.

**Delete one.**
**Expect:** A confirmation dialog, then it disappears.

## 10. API and Swagger checks

Open `http://localhost:4000/api/docs`. Useful calls:

```
curl -s localhost:4000/api/health
curl -s -X POST localhost:4000/api/knowledge/search -H 'content-type: application/json' \
  -d '{"query":"hotel budget per night in London","topK":3,"mode":"hybrid"}'
curl -s -X POST localhost:4000/api/agent/turn -H 'content-type: application/json' \
  -d '{"text":"How many PTO days do I have left this year?"}'
curl -s localhost:4000/api/tools
curl -s localhost:4000/api/tools/usage
curl -s localhost:4000/api/hr/tickets
curl -s localhost:4000/api/conversations
```

The agent-turn response contains the assistant message with `sources`, `steps` (the trace) and `metrics` (latency, tokens, cost), which is the easiest way to script checks.

## 11. Automated tests

| COMMAND | WHAT RUNS | NEEDS |
|---|---|---|
| `pnpm test` | Unit tests for the API, the web app and the contracts package (84 tests) | nothing |
| `pnpm --filter @voice-agent/api test:e2e` | API end-to-end incl. the Socket.IO gateway (20 tests), mock providers | Postgres + Redis running |
| `pnpm --filter @voice-agent/web test:e2e` | Browser suite (7 tests) incl. a hands-free voice turn with a fake microphone | API + web running (or it starts them); mock or OpenAI mode |
| `pnpm eval:retrieval` | Retrieval quality report (hit@1, hit@3, MRR per mode) | Postgres; OpenAI key for real embeddings |
| `pnpm --filter @voice-agent/api bench:speech` | Speech latency micro-benchmark | OpenAI key (costs well under a cent) |

> The browser suite run against an OpenAI-mode API spends about $0.03 per run. CI always runs in mock mode.

# 12. Troubleshooting

| SYMPTOM | CAUSE AND FIX |
| --- | --- |
| Badge says "disconnected" | The API is not running on port 4000. `pnpm dev`, or check `http://localhost:4000/api/health`. |
| Orb disabled / "Microphone access is not available" | Browser blocked the microphone or the page is not on localhost/https. Allow it in site settings and reload. |
| Hands-free never triggers | Speak clearly for at least half a second; the detector ignores very short sounds. Check the input device in the browser's site settings. |
| The agent interrupts itself | Speaker audio feeding back into the microphone. Use headphones or Push to talk. |
| Spoken replies are a chime, transcripts are demo phrases | Mock mode: no key found. Add `OPENAI_API_KEY` to `.env`, restart the API, run `pnpm db:seed` to re-embed. |
| Answers cite nothing / "weak" grades everywhere | Index embedded with a different provider. Run `pnpm db:seed` (documents remember their embedding model). |
| Port already allocated | Another Postgres uses 5434 or Redis 6379: change the host port in `docker/docker-compose.yml` and `DATABASE_URL` / `REDIS_URL`. |
| Reindex jobs stay queued | Redis is down or the API worker is not running. `pnpm infra:up` and restart the API. |